

```
import os
import numpy as np
import pandas as pd
#import matplotlib.pyplot as plt
```

✓ Google Drive Mount

1. 네이버 영화 리뷰 데이터 다운로드

네이버 영화 리뷰 데이터는 GitHub에서 제공되는 NSMC(Naver Sentiment Movie Corpus) 데이터를 사용할 수 있습니다. 이 데이터는 긍정(1) 또는 부정(0)으로 레이블된 리뷰로 구성되어 있습니다

```
#internet resource
import urllib.request
```

```
# 네이버 영화 리뷰 데이터 다운로드
```

```
urllib.request.urlretrieve("https://raw.githubusercontent.com/e9t/nsmc/master/ratings_train.txt",
urllib.request.urlretrieve("https://raw.githubusercontent.com/e9t/nsmc/master/ratings_test.txt", f
```

```
➡ ('/content/drive/MyDrive/Takeout/드라이브/산업기술대학교/2024년 강의자료/자연어처
리/ratings_test.txt',
<http.client.HTTPMessage at 0x7b2cf23dbeb0>)
```

```
train_data_path = "/content/drive/MyDrive/Takeout/드라이브/산업기술대학교/2024년 강의자료/자연어처
test_data_path = "/content/drive/MyDrive/Takeout/드라이브/산업기술대학교/2024년 강의자료/자연어처
```

```
train_naver_review = pd.read_table(train_data_path)
test_naver_review = pd.read_table(test_data_path)
```

```
# 데이터 확인
```

```
print(f'훈련용 리뷰 개수: {len(train_naver_review)}')
print(f'테스트용 리뷰 개수: {len(test_naver_review)}')
```

```
# 상위 5개 샘플 출력
```

```
print(train_naver_review.head())
```

```
➡ 훈련용 리뷰 개수: 150000
테스트용 리뷰 개수: 50000
```

	id	document	label
0	9976970	아 더빙.. 진짜 짜증나네요 목소리	0
1	3819312	흠...포스터보고 초딩영화줄....오버연기조차 가볍지 않구나	1
2	10265843	너무재밌었다그래서보는것을추천한다	0
3	9045019	교도소 이야기구먼 ..솔직히 재미는 없다..평점 조정	0
4	6483659	사이몬페그의 익살스런 연기가 돋보였던 영화!스파이더맨에서 늙어보이기만 했던 커스	

✓ 2. 텍스트 전처리

데이터를 불러온 후, 텍스트 데이터를 정제하는 과정이 필요합니다. 여기서는 ##### 불필요한 특수문자 제거, 중복 제거 등의 작업을 수행합니다.

```
import re

# 1. 중복 제거
train_naver_review.drop_duplicates(subset=['document'], inplace=True)

# 2. Null 값 제거
train_naver_review = train_naver_review.dropna(how='any')

# 3. 한글과 공백을 제외한 모든 문자 제거
train_naver_review['document'] = train_naver_review['document'].apply(lambda x: re.sub(r'[^ㄱ-ㅎㅏ-ㅣ가-힣 ]', ''))

# 전처리 후 데이터 확인
print(train_naver_review.head())
```

	id	document	label
0	9976970	아 더빙 진짜 짜증나네요 목소리	0
1	3819312	흠포스터보고 초딩영화줄오버연기조차 가볍지 않구나	1
2	10265843	너무재밌었다그래서보는것을추천한다	0
3	9045019	교도소 이야기구먼 솔직히 재미는 없다평점 조정	0
4	6483659	사이몬페그의 익살스런 연기가 돋보였던 영화스파이더맨에서 늙어보이기만 했던 커스틴	

이 코드는 ratings_train.txt와 ratings_test.txt 파일을 다운로드하여 train_data와 test_data로 불러옵니다. 각 파일에는 id, document, label 세 가지 열이 있으며, 여기서 document는 영화 리뷰 텍스트, label은 긍정(1) 또는 부정(0)을 나타냅니다

✓ 3.

문장 토큰화 및 품사 태깅

한국어 텍스트를 처리하기 위해서는 형태소 분석이 필요합니다. 이를 위해 KoNLPy 라이브러리의 Okt 형태소 분석기를 사용하여 문장을 토큰화하고, 품사 태깅을 통해 필요한 단어만 추출할 수 있습니다.

```
# 1. GitHub에서 Mecab-ko-for-Google-Colab 프로젝트 클론
!git clone https://github.com/SOMJANG/Mecab-ko-for-Google-Colab.git
# 2. 해당 디렉토리로 이동
%cd Mecab-ko-for-Google-Colab
```

3. 설치 스크립트 실행

!bash install_mecab-ko_on_colab_light_220429.sh



Cloning into 'Mecab-ko-for-Google-Colab'...

remote: Enumerating objects: 138, done.

remote: Counting objects: 100% (47/47), done.

remote: Compressing objects: 100% (38/38), done.

remote: Total 138 (delta 26), reused 22 (delta 8), pack-reused 91 (from 1)

Receiving objects: 100% (138/138), 1.72 MiB | 16.89 MiB/s, done.

Resolving deltas: 100% (65/65), done.

/content/Mecab-ko-for-Google-Colab

Installing konlpy.....

Collecting konlpy

Downloading konlpy-0.6.0-py2.py3-none-any.whl.metadata (1.9 kB)

Collecting JPype1>=0.7.0 (from konlpy)

Downloading JPype1-1.5.0-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata

Requirement already satisfied: lxml>=4.1.0 in /usr/local/lib/python3.10/dist-packages (from JPype1)

Requirement already satisfied: numpy>=1.6 in /usr/local/lib/python3.10/dist-packages (from JPype1)

Requirement already satisfied: packaging in /usr/local/lib/python3.10/dist-packages (from JPype1)

Downloading konlpy-0.6.0-py2.py3-none-any.whl (19.4 MB)

19.4/19

Downloading JPype1-1.5.0-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (488 kB)

488.6/4

Installing collected packages: JPype1, konlpy

Successfully installed JPype1-1.5.0 konlpy-0.6.0

Done

Installing mecab-0.996-ko-0.9.2.tar.gz.....

Downloading mecab-0.996-ko-0.9.2.tar.gz.....

from <https://bitbucket.org/eunjeon/mecab-ko/downloads/mecab-0.996-ko-0.9.2.tar.gz>--2024-11-05 04:21:13-- <https://bitbucket.org/eunjeon/mecab-ko/downloads/mecab-0.996-ko-0.9.2.tar.gz>

Resolving bitbucket.org (bitbucket.org)... 104.192.142.26, 104.192.142.24, 104.192.142.25,

Connecting to bitbucket.org (bitbucket.org)|104.192.142.26|:443... connected.

HTTP request sent, awaiting response... 302 Found

Location: <https://bbuseruploads.s3.amazonaws.com/eunjeon/mecab-ko/downloads/mecab-0.996-ko-0.9.2.tar.gz>--2024-11-05 04:21:14-- <https://bbuseruploads.s3.amazonaws.com/eunjeon/mecab-ko/downloads/mecab-0.996-ko-0.9.2.tar.gz>

Resolving bbuseruploads.s3.amazonaws.com (bbuseruploads.s3.amazonaws.com)... 52.217.139.1,

Connecting to bbuseruploads.s3.amazonaws.com (bbuseruploads.s3.amazonaws.com)|52.217.139.1|:

HTTP request sent, awaiting response... 200 OK

Length: 1414979 (1.3M) [application/x-tar]

Saving to: 'mecab-0.996-ko-0.9.2.tar.gz'

mecab-0.996-ko-0.9. 100%[=====>] 1.35M --.-KB/s in 0.1s

2024-11-05 04:21:14 (10.5 MB/s) - 'mecab-0.996-ko-0.9.2.tar.gz' saved [1414979/1414979]

Done

Unpacking mecab-0.996-ko-0.9.2.tar.gz.....

Done

Change Directory to mecab-0.996-ko-0.9.2.....

installing mecab-0.996-ko-0.9.2.tar.gz.....

configure

make

make check

make install

ldconfig

Done

Change Directory to /content

Downloading mecab-ko-dic-2.1.1-20180720.tar.gz.....

from <https://bitbucket.org/eunjeon/mecab-ko-dic/downloads/mecab-ko-dic-2.1.1-20180720.tar.gz>--2024-11-05 04:23:24-- <https://bitbucket.org/eunjeon/mecab-ko-dic/downloads/mecab-ko-dic-2.1.1-20180720.tar.gz>

```
import konlpy
from konlpy.tag import Mecab
```

```
mecab = Mecab()
```

```
# 전체 리뷰 데이터를 대상으로 형태소 분석 및 품사 태깅 수행
```

```
def tokenize_and_pos_tagging(text):
    # Null 값이 들어올 경우 빈 리스트 반환
    if not isinstance(text, str):
        return []
    return mecab.pos(text) # 각 문장에 대해 품사 태깅
```

```
# 전체 데이터에 대해 형태소 분석 및 품사 태깅 적용
```

```
train_naver_review['tokenized_pos'] = train_naver_review['document'].apply(tokenize_and_pos_tagging)
```

```
# 결과 확인 (상위 5개의 리뷰에 대해 형태소 및 품사 태깅 결과 출력)
```

```
print(train_naver_review[['document', 'tokenized_pos']].head())
```

```

      document      tokenized_pos
0  [(아, IC), (더, MAG), (빙, MAG), (진짜, MAG), (짜증, ...
1  [(흠, IC), (포스터, NNP), (보고, JKB), (초딩, NNG), (영...
2  [(너무, MAG), (재, XPN), (말었다그래서보는것을추천한다, UNKNOWN)]
3  [(교도소, NNG), (이야기, NNG), (구만, VCP+EF), (솔직히, M...
4  [(사이몬페그, NNP), (의, JKB), (익살, NNG), (스런, XSA+E...
```

```
# 결과 확인 (상위 5개의 리뷰에 대해 형태소 및 품사 태깅 결과 출력)
```

```
train_naver_review['tokenized_pos'][0]
```

```

[('아', 'IC'),
 ('더', 'MAG'),
 ('빙', 'MAG'),
 ('진짜', 'MAG'),
 ('짜증', 'NNG'),
 ('나', 'NP'),
 ('네요', 'VCP+EC'),
 ('목소리', 'NNG')]

```

```
# 형용사와 동사만 추출하는 함수 정의
```

```
def extract_adj_verb(pos_tags):
    return [word for word, tag in pos_tags if tag in ['VV', 'VA', 'VCP', 'VCN']]
```

```
# 전체 데이터에서 형용사와 동사만 추출하여 새로운 열로 저장
```

```
train_naver_review['adj_verb_tokens'] = train_naver_review['tokenized_pos'].apply(extract_adj_verb)
```

```
# 결과 확인 (상위 5개의 리뷰에서 형용사와 동사만 추출한 결과 출력)
```

```
print(train_naver_review[['document', 'adj_verb_tokens']].head())
```

```

      document adj_verb_tokens
0  아 더빙 진짜 짜증나네요 목소리      []
1  흠포스터보고 초딩영화줄오버연기조차 가볍지 않구나      [가볍]
2  너무재말었다그래서보는것을추천한다      []
3  교도소 이야기구만 솔직히 재미는 없다평점 조정      [없]
4  사이몬페그의 익살스런 연기가 돋보였던 영화스파이더맨에서 늙어보이기만 했던 커스틴 던...
```

```
# 형용사와 동사만 추출하는 함수 정의
def extract_adj_verb(pos_tags):
    return [word for word, tag in pos_tags if tag in ['VV', 'VA', 'VCP', 'VCN', 'VX', 'NNG']]

# 전체 데이터에서 형용사와 동사만 추출하여 새로운 열로 저장
train_naver_review['adj_verb_tokens'] = train_naver_review['tokenized_pos'].apply(extract_adj_verb)

# 결과 확인 (상위 5개의 리뷰에서 형용사와 동사만 추출한 결과 출력)
print(train_naver_review[['document', 'adj_verb_tokens']].head())
```



```
document W
0      아 더빙 진짜 짜증나네요 목소리
1      흥포스터보고 초딩영화줄오버연기조차 가볍지 않구나
2      너무재밌었다그래서보는것을추천한다
3      교도소 이야기구먼 솔직히 재미는 없다평점 조정
4  사이몬페그의 익살스런 연기가 돋보였던 영화스파이더맨에서 늙어보이기만 했던 커스틴 던...

adj_verb_tokens
0      [짜증, 목소리]
1  [초딩, 영화, 줄, 오버, 연기, 가볍, 않]
2      []
3  [교도소, 이야기, 재미, 없, 평점, 조정]
4      [익살, 연기, 영화, 늙, 보이]
```

✓ 4. TF-IDF 벡터화 및 분류 모델 학습

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# adj_verb_tokens 리스트를 문자열로 변환 (빈 리스트는 공백 문자열로 처리)
train_naver_review['adj_verb_str'] = train_naver_review['adj_verb_tokens'].apply(lambda x: ' '.join(x))

# 결과 확인 (상위 5개의 리뷰에서 형용사와 동사만 추출한 결과 출력)
print(train_naver_review[['document', 'adj_verb_tokens', 'adj_verb_str']].head())

# TF-IDF 벡터화 (형용사와 동사가 포함된 토큰 리스트를 문자열로 변환하여 벡터화)
tfidf_vectorizer = TfidfVectorizer(max_features=1000)

# TF-IDF 벡터화 수행
X_train_tfidf = tfidf_vectorizer.fit_transform(train_naver_review['adj_verb_str'])

# 결과 확인
print("TF-IDF 벡터화 완료. 데이터 크기:", X_train_tfidf.shape)
```



```
document W
0      아 더빙 진짜 짜증나네요 목소리
1      흥포스터보고 초딩영화줄오버연기조차 가볍지 않구나
2      너무재밌었다그래서보는것을추천한다
```

6/10

Name: 0, dtype: float64

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# 라벨 데이터 준비 (긍정/부정 레이블이 있는 열)
y = train_naver_review['label'] # 예: 긍정(1), 부정(0)

# 학습 데이터와 테스트 데이터 분리 (80% 학습, 20% 테스트)
X_train, X_test, y_train, y_test = train_test_split(tfidf_df, y, test_size=0.2, random_state=42)

# Random Forest 분류기 생성 및 학습
rf_model = RandomForestClassifier(n_estimators=10, random_state=42)
rf_model.fit(X_train, y_train)

# 테스트 데이터로 예측 수행
y_pred = rf_model.predict(X_test)

# 정확도 평가
accuracy = accuracy_score(y_test, y_pred)
print(f"Random Forest 정확도: {accuracy:.4f}")
```

➡ Random Forest 정확도: 0.7209

```
from gensim.models import Word2Vec

# Word2Vec 모델 학습 (CBOW 방식)
sentences = train_naver_review['adj_verb_tokens'].tolist() # 리뷰에서 추출한 형용사와 동사 토큰
word2vec_model = Word2Vec(sentences, vector_size=100, window=5, min_count=1, sg=0) # sg=0이면 CBC

print(sentences)
```

➡ [['짜증', '목소리'], ['초딩', '영화', '줄', '오버', '연기', '가볍', '않'], [], ['교도소', '이

```
# 각 리뷰 문장을 벡터로 변환 (평균 벡터 사용)
def get_sentence_vector(model, sentence):
    vectors = [model.wv[word] for word in sentence if word in model.wv]
    return sum(vectors) / len(vectors) if vectors else [0] * model.vector_size

train_naver_review['vector'] = train_naver_review['adj_verb_tokens'].apply(lambda x: get_sentence_

train_naver_review['adj_verb_tokens']
```




adj_verb_tokens

0	[짜증, 목소리]
1	[초딩, 영화, 줄, 오버, 연기, 가볍, 얇]
2	[]
3	[교도소, 이야기, 재미, 없, 평점, 조정]
4	[익살, 연기, 영화, 늙, 보이]
...	...
149995	[인간, 문제지, 소, 죄]
149996	[평점, 낮]
149997	[한국인, 거들먹거리, 혼혈, 착하]
149998	[청춘, 영화, 방황, 우울, 날, 자화상]
149999	[영화, 최초, 수간, 내용, 영화]

146182 rows × 1 columns

dtype: object

train_naver_review['adj_verb_tokens'][0]



['짜증', '목소리']

'짜증' in word2vec_model.wv



True

word2vec_model.wv['짜증']



```
array([ 0.46691826, -0.19899252,  0.25916556,  1.1886744 ,  0.24757668,
        -0.50282943,  0.09414975,  0.09983738, -1.8252503 , -1.4518286 ,
        -0.8542363 , -0.9872464 ,  0.7144678 ,  1.5052831 , -0.36110026,
         0.6072812 , -1.2255896 , -0.45451078, -0.38652372, -0.13339928,
         0.8490565 ,  1.2702326 ,  0.51050353, -0.35542324,  0.2909641 ,
         1.2272997 , -0.7865383 , -1.9346312 , -0.36399606,  0.8936358 ,
         0.01530191,  0.20666479,  0.03042859, -1.1826364 ,  1.2579716 ,
         2.1383474 , -1.8724072 ,  0.65086186, -0.14251435, -2.41082 ,
        -1.0217351 , -0.40718532, -0.08478332, -0.26395005,  0.9057805 ,
        -0.34404328,  2.2809925 , -0.36827463,  1.4582367 , -1.4405823 ,
         1.2706635 , -0.49193898, -1.7814232 , -0.9394597 ,  0.2658669 ,
        -0.9714006 ,  0.48523223, -1.0460317 ,  1.6888767 , -0.2364174 ,
         0.98363185, -0.92453694, -1.3263803 ,  0.35157433, -1.4005188 ,
         1.4978433 , -0.09992933,  0.7590579 , -0.40499288,  2.1376376 ,
         0.1611998 , -0.25094697,  0.48670763,  1.3535489 , -0.67935264,
         0.26791817, -1.585499 , -1.3065755 ,  0.18680525,  0.29038405,
        -0.10593953,  0.1732983 , -2.1096501 ,  0.6288732 ,  0.11917756,
        -0.8863292 , -0.3846705 ,  1.1672782 ,  0.8340228 ,  0.6911225 ,
         0.1559087 , -0.28269368,  0.45575473,  0.22738244,  0.03286204,
```

```
0.51058525, 0.15828432, -2.2140203 , 1.5270276 , -1.5803531 ],
dtype=float32)
```

```
# 라벨 데이터 준비
```

```
y = train_naver_review['label']
```

```
# 벡터 데이터를 리스트로 변환
```

```
X = train_naver_review['vector'].tolist()
```

```
# 학습 데이터와 테스트 데이터 분리
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# Random Forest 분류기 생성 및 학습
```

```
rf_model = RandomForestClassifier(n_estimators=10, random_state=42)
```

```
rf_model.fit(X_train, y_train)
```

```
# 테스트 데이터로 예측 수행
```

```
y_pred = rf_model.predict(X_test)
```

```
# 정확도 평가
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print(f"Random Forest 정확도: {accuracy:.4f}")
```



Random Forest 정확도: 0.7435

"Pre-Trained Model 사용가능"